



Optimism: L2 Interop Re-review

Security Review

Cantina Managed review by:

R0bert, Lead Security Researcher

Sujith Somraaj, Lead Security Researcher

June 18, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	<code>sendETH</code> accepts invalid destination chain IDs	4
3.1.2	Relay value is not source-authenticated	4
3.1.3	<code>RelayMessage</code> consumes no-code target messages	5
3.1.4	Unbounded target <code>returnData_</code> in <code>relayMessage</code> enables relayer gas griefing	5
3.2	Informational	6
3.2.1	Cross-domain context is readable by nested callees during relay	6
3.2.2	<code>CrossL2Inbox</code> trusts client access-list validation	7
3.2.3	<code>RelayETH</code> source validation depends on offchain dependency-set enforcement	7
3.2.4	Direct <code>CrossL2Inbox</code> consumers can replay messages	8
3.2.5	<code>CrossL2Inbox</code> does not bind destination or application	8
3.2.6	<code>SafeSend</code> self-recipient burns value	9
3.2.7	Warm-read threshold depends on gas schedule	9
3.2.8	<code>notEntered</code> requires an entered call-depth context	9
3.2.9	<code>CrossL2Inbox</code> target filtering does not match documentation	10
3.2.10	<code>ReentrantAware</code> can leave call depth incremented after inline assembly termination	10
3.2.11	Call-depth wraparound comment overstates safety	11
3.2.12	<code>ETHLiquidity.mint</code> underfunding has a generic revert	11
3.2.13	L2-to-L2 relay does not validate message version	12
3.2.14	<code>OnlyEntered</code> relies on the reentrancy guard instead of metadata state	12
3.2.15	Output root version is not constrained	13
3.2.16	Prefunded <code>SafeSend</code> addresses inject extra value	13
3.2.17	<code>ResendMessage</code> can spam duplicate valid message logs	14
3.2.18	<code>sendETH</code> accepts zero-value bridge messages	14
3.2.19	Hand-maintained <code>SentMessage</code> selector can drift from the event layout	14
3.2.20	<code>ETHLiquidity</code> invariant checks the wrong balance	15
3.2.21	<code>ETHLiquidity</code> events record only <code>msg.sender</code> (always the bridge), losing the initiator	15
3.2.22	<code>L2ToL2CrossDomainMessenger</code> <code>sendMessage</code> documentation contains stale legacy messenger warnings	16
3.2.23	<code>L2ToL2CrossDomainMessenger</code> redeclares errors inherited from <code>TransientReentrancyAware</code>	16
3.2.24	<code>CrossL2Inbox</code> validation failure error differs from the spec reference implementation	16
3.2.25	<code>CrossL2Inbox</code> spec requires <code>Identifier</code> getters that are not implemented by the stateless design	17

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Optimism is a fast, stable, and scalable L2 blockchain built by Ethereum developers, for Ethereum developers. Built as a minimal extension to existing Ethereum software, Optimism's EVM-equivalent architecture scales your Ethereum apps without surprises. If it works on Ethereum, it works on Optimism at a fraction of the cost.

From Jun 1st to Jun 3rd the Cantina team conducted a review of [optimism](#) on commit hash [fa9974a2](#). The team identified a total of **29** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	1	3
Gas Optimizations	0	0	0
Informational	25	8	17
Total	29	9	20

2.1 Scope

The security review had the following components in scope for [optimism](#) on commit hash [fa9974a2](#):

```
packages/contracts-bedrock/src
├── L2
│   ├── CrossL2Inbox.sol
│   ├── ETHLiquidity.sol
│   ├── L2ToL2CrossDomainMessenger.sol
│   └── SuperchainETHBridge.sol
├── libraries
│   ├── Hashing.sol
│   └── TransientContext.sol
└── universal
    └── SafeSend.sol
```

3 Findings

3.1 Low Risk

3.1.1 `sendETH` accepts invalid destination chain IDs

Severity: Low Risk

Context: `SuperchainETHBridge.sol#L41-L58`

Description: `SuperchainETHBridge.sendETH` only checks that `_to` is not the zero address. It then moves the caller's ETH into `ETHLiquidity` and asks `L2ToL2CrossDomainMessenger` to emit a message for the caller-supplied `_chainId`.

The bridge never checks whether that chain ID is part of the interop dependency set or whether it names a real destination chain. The messenger only rejects `block.chainid`, so any other value, including `0` or an arbitrary unused chain ID, can be recorded as the destination.

This is a problem because `sendETH` moves value before the destination is known to be relayable. A user, wallet or SDK can send ETH to a nonexistent or unsupported chain ID, the source-chain transaction will still succeed and the ETH will remain in the source chain's liquidity contract. No valid destination chain can then relay the message to the intended recipient.

Recommendation: Validate `_chainId` before moving ETH into `ETHLiquidity`. If the bridge cannot query an on-chain dependency-set registry or precompile, enforce the supported destination set in the UI and SDK and document that guard as required for safe use. Once the protocol exposes an on-chain registry, bind `sendETH` to it so unsupported destinations are rejected before user value is locked.

OP Labs: Acknowledged. There is no onchain dependency set so validating `chainId` should be happening on the UI/offchain side.

Cantina Managed: Acknowledged.

3.1.2 Relay value is not source-authenticated

Severity: Low Risk

Context: `L2ToL2CrossDomainMessenger.sol#L203-L245`

Description: `relayMessage` is payable and forwards `msg.value` to the target, but the source-chain message hash does not commit to any ETH value. The committed message only includes destination, source, nonce, sender, target and calldata.

This means the relayer can choose the ETH value attached to a valid message at relay time. The relayer funds that ETH, so this does not let the messenger steal value. It can still change the behavior of targets that treat `msg.value` as part of the source-authenticated command.

For example, a payable receiver that credits value to the cross-domain sender would be accepting relayer-supplied ETH rather than source-committed ETH.

Recommendation: Receiver contracts should reject unexpected ETH with `require(msg.value == 0)` unless they deliberately support relayer-funded calls. If L2-to-L2 value-bearing messages are intended, include the value in the committed message format and compute replay protection over that value.

OP Labs: Acknowledged. Generic `L2ToL2CrossDomainMessenger` messages are not intended to carry source-authenticated ETH value. Applications that need ETH movement should use `SuperchainETHBridge`, which derives the bridged amount from the message payload and liquidity flow rather than from relayer-supplied `msg.value`.

Cantina Managed: Acknowledged.

3.1.3 RelayMessage consumes no-code target messages

Severity: Low Risk

Context: L2ToL2CrossDomainMessenger.sol#L241-L245

Description: relayMessage marks a message as successful before it calls the decoded target. It then treats any low-level call that returns success == true as a completed delivery.

Low-level EVM calls to EOAs and not-yet-deployed addresses return success even though no contract code ran. Consequently, any relayer can permanently consume a valid message whose target is a counterfactual address or an address that is expected to be deployed later. Once successfulMessages[messageHash] is set, the same message cannot be relayed again after the real target contract is deployed.

This is not a direct issue for the current SuperchainETHBridge transfer, because the bridge targets the destination bridge predeploy and expects it to exist before interop is active. The risk applies to general messenger users that send messages to deterministic deployment addresses, cross-chain account contracts, governance executors, routers or other contracts that may not have code at relay time.

Recommendation: If L2ToL2CrossDomainMessenger is intended for contract calls, reject decoded targets without runtime code before marking the message successful.

```
if (target.code.length == 0) revert TargetHasNoCode();
```

If EOA or value-transfer targets are intentionally supported, split that behavior from contract-message delivery so a no-code call cannot silently consume a contract-bound message. At minimum, document that messages to no-code targets can be permanently consumed by any relayer.

OP Labs: Fixed in commit da3c4f4.

Cantina Managed: Fix verified. The accepted remediation was documentation: both the implementation and interface now state that relaying to a no-code target is a no-op success that permanently consumes the message, so integrators are explicitly warned about the behavior.

3.1.4 Unbounded target returnData_ in relayMessage enables relayer gas griefing

Severity: Low Risk

Context: L2ToL2CrossDomainMessenger.sol#L245

Description: L2ToL2CrossDomainMessenger.relayMessage calls the message target with:

```
(success, returnData_) = target.call{ value: msg.value }(message);
```

The target and calldata are user-controlled through sendMessage. Unlike other OP Stack relay paths that use SafeCall and intentionally avoid copying returndata, this call copies the entire target returndata into memory. On success it hashes the full buffer for RelayedMessage, and on failure it bubbles the full revert data.

A malicious target can return or revert with very large returndata, forcing relayers to pay excessive memory expansion, copy, and hashing costs. The impact is relayer gas griefing.

Recommendation: Avoid copying unbounded target returndata in relayMessage. Use an assembly call or SafeCall-style helper that discards returndata, or cap the copied returndata to a small bounded prefix. If return data is needed for the API/event, emit or return only a bounded value, such as a capped prefix plus a hash.

OP Labs: Acknowledged. Relayers would usually simulate before submitting and are not forced to relay.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Cross-domain context is readable by nested callees during relay

Severity: Informational

Context: `L2ToL2CrossDomainMessenger.sol#L241-L245`

Description: `relayMessage` stores the source chain and source sender in transient storage before calling the message target. The public context getters only check that the messenger is currently entered. They do not require the caller to be the direct target of the relayed message.

Consequently, a malicious target can call another contract while the context is set. If that downstream contract authorizes actions only by reading `crossDomainMessageContext()` and does not also require `msg.sender == L2_T0_L2_CROSS_DOMAIN_MESSENGER`, it can mistake the nested call for a direct cross-domain relay.

This is an application-facing authorization hazard rather than a direct replay bug in the messenger. `SuperchainETHBridge` follows the safer pattern by checking the local messenger caller before trusting the context.

Realistic Example Flow: Assume an application has two chains:

1. Chain A has a trusted contract named `RemoteController`.
2. Chain B has a receiver named `RewardsVault`.
3. `RewardsVault` is supposed to release funds only when `RemoteController` sends it a cross-chain message through `L2ToL2CrossDomainMessenger`.

The intended safe flow is:

1. `RemoteController` on chain A calls `sendMessage()` on the source-chain messenger.
2. The message says: destination is chain B, target is `RewardsVault`, calldata is `releaseReward(user, amount)`.
3. On chain B, `relayMessage()` validates the source event through `CrossL2Inbox`.
4. `relayMessage()` stores the active cross-domain context: source chain is chain A and cross-domain sender is `RemoteController`.
5. The messenger calls `RewardsVault.releaseReward(...)`.
6. A safe `RewardsVault` checks both facts:

```
if (msg.sender != Predeploys.L2_T0_L2_CROSS_DOMAIN_MESSENGER) revert Unauthorized();  
  
(address sender, uint256 source) = messenger.crossDomainMessageContext();  
if (sender != remoteController || source != chainA) revert Unauthorized();
```

The unsafe flow appears when a receiver skips the local caller check:

```
(address sender, uint256 source) = messenger.crossDomainMessageContext();  
if (sender != remoteController || source != chainA) revert Unauthorized();  
  
_releaseReward(recipient, amount);
```

Now consider a different valid relay from `RemoteController` where the target is not `RewardsVault`, but an attacker-controlled or attacker-influenced contract named `AttackerTarget`. This can happen if the trusted remote contract is a router, executor, bridge adapter or app contract that can be made to call untrusted destination code.

1. A valid chain A message from `RemoteController` is relayed to `AttackerTarget` on chain B.

2. During `relayMessage()`, the messenger stores the same kind of active context: source chain is chain A and cross-domain sender is `RemoteController`.
3. The messenger calls `AttackerTarget`.
4. Before `relayMessage()` clears the context, `AttackerTarget` calls `RewardsVault.releaseReward(attacker, amount)`.
5. Inside `RewardsVault`, `msg.sender` is `AttackerTarget`, not the messenger.
6. However, `RewardsVault` can still call `crossDomainMessageContext()` because the messenger is still entered.
7. The context getter returns the active relay context. If that context matches what `RewardsVault` trusts, the context-only authorization passes.
8. `RewardsVault` releases funds even though it was not the direct target of the cross-chain message.

The impact is therefore not that any user can forge cross-chain context at any time. The impact is that context can be borrowed during a valid relay. Any downstream contract that treats `crossDomainMessageContext()` as proof of a direct cross-chain call can authorize the wrong caller. Depending on the downstream contract, this can become unauthorized minting, withdrawals, role changes or configuration changes.

Recommendation: Document that receiver contracts must require `msg.sender == Predeploys.L2_TO_L2_CROSS_DOMAIN_MESSENGER` before trusting `crossDomainMessageSender`, `crossDomainMessageSource` or `crossDomainMessageContext`.

OP Labs: Acknowledged. This is documented here in predeploys.html#sending-messages but wouldn't hurt to add an extra comment on the code as well.

Cantina Managed: Acknowledged.

3.2.2 `CrossL2Inbox` trusts client access-list validation

Severity: Informational

Context: [CrossL2Inbox.sol#L68-L82](#)

Description: `validateMessage` only checks whether the calculated checksum storage slot is warm, then emits `ExecutingMessage`. It does not prove that the source-chain log exists or that the access-list entry was derived from a valid initiating message.

This is safe only when OP clients enforce interop access-list validation before accepting or deriving the transaction. In a raw EVM environment, a caller can warm arbitrary storage slots through an EIP-2930 access list. Consequently, running this predeploy without the interop filter or with the filter configured to pass through entries, would allow fabricated message identifiers to pass the Solidity check.

This is a deployment and client-configuration assumption, not a confirmed Solidity bug under the expected OP Stack interop threat model.

Recommendation: Treat interop access-list validation as a mandatory consensus and admission invariant for every chain that deploys `CrossL2Inbox`. Production clients should reject passthrough filter modes and reject transactions with unchecked `CrossL2Inbox` access-list entries, and include regression tests proving that fabricated checksum entries are rejected before execution.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.3 `ReLayETH` source validation depends on offchain dependency-set enforcement

Severity: Informational

Context: [SuperchainETHBridge.sol#L67-L70](#)

Description: `relayETH` accepts a cross-chain ETH release when the local messenger reports that the cross-domain sender is `address(this)`. The bridge does not validate the source chain ID against an on-chain dependency-set registry.

The messenger and inbox perform the lower-level message checks. `L2ToL2CrossDomainMessenger.relayMessage` verifies that the log came from the messenger pre-deploy and asks `CrossL2Inbox` to validate the message. `CrossL2Inbox` then checks that the checksum storage slot was included in the access list. The code comments state that nodes pre-check message validity before execution.

Therefore, the bridge's safety depends on the supervisor and node layer rejecting messages from unsupported, misconfigured or malicious source chains. If that external enforcement is bypassed or misconfigured, a source chain that can produce a message from the bridge predeploy address can cause destination-chain ETH to be released from `ETHliquidity`.

Recommendation: Document this as an explicit bridge invariant and monitor dependency-set configuration at activation and upgrade time.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.4 Direct `CrossL2Inbox` consumers can replay messages

Severity: Informational

Context: [CrossL2Inbox.sol#L68-L82](#)

Description: `validateMessage` has no replay state. Once a valid checksum is present in the transaction access list, the same identifier and message hash can be validated more than once. The function can also be called repeatedly in one transaction after the checksum slot is warm.

The canonical `L2ToL2CrossDomainMessenger` adds replay protection with `successfulMessages`, so this is not a bug in that integration. The risk applies to applications that call `CrossL2Inbox` directly and treat a successful validation as a one-time execution guarantee. Those applications can double-process the same valid message unless they store and check their own consumed-message state.

Recommendation: Direct consumers should persist a consumed-message key before executing message effects. The key should include the source chain ID, source log identifier, and application-level message hash. Applications that do not need custom message handling should use `L2ToL2CrossDomainMessenger` instead of calling `CrossL2Inbox` directly.

OP Labs: Acknowledged. Applications should handle replay protection in this scenario; having a consumed-message key could limit the needs of some consumers.

Cantina Managed: Acknowledged.

3.2.5 `CrossL2Inbox` does not bind destination or application

Severity: Informational

Context: [CrossL2Inbox.sol#L84-L112](#)

Description: `calculateChecksum` binds the source origin, source block number, source timestamp, source log index, source chain ID and message hash. It does not bind the destination chain, destination contract, message schema or application-specific nonce unless those values are already included in `_msgHash`.

The canonical `L2ToL2CrossDomainMessenger` handles this separately by decoding the sent message, requiring the destination to equal `block.chainid` and hashing destination, source, nonce, sender, target and message body before replay checks. Direct `CrossL2Inbox` consumers do not get those protections automatically. If an application hashes only a portable payload, the same validated source event can be accepted by the wrong destination or by a different application that uses the same payload format.

Recommendation: Direct consumers should domain-separate their message hash. Include the destination chain ID, destination contract, source chain ID, source sender, application schema version, nonce and payload in the data committed by `_msgHash`. Prefer the canonical messenger when the application does not need custom validation.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.6 `SafeSend` self-recipient burns value

Severity: Informational

Context: `SafeSend.sol#L8-L10`

Description: `SafeSend` does not check whether `_recipient` is the temporary contract that is being created. Because a `CREATE` address is deterministic from the creator address and nonce, a caller can precompute the next `SafeSend` address and pass that address as the recipient.

In that case, the constructor executes `selfdestruct(address(this))`. The value is not delivered to a usable account; it is destroyed with the temporary contract.

This is not a practical third-party theft vector in the reviewed callers because the recipient is chosen by the user or by a privileged actor. It is still a hidden burn case. In `SuperchainETHBridge`, it also bypasses the existing zero-address check because the computed temporary address is nonzero.

Recommendation: Document this edge case as equivalent to choosing an inaccessible recipient.

OP Labs: Fixed in commit `3bf3026`.

Cantina Managed: Fix verified. The issue was remediated as documentation, which matched the recommendation. `SafeSend` now explicitly warns that choosing the temporary contract itself as the recipient burns the value in the same way as choosing any inaccessible recipient.

3.2.7 Warm-read threshold depends on gas schedule

Severity: Informational

Context: `CrossL2Inbox.sol#L114-L129`

Description: `_isWarm` classifies a checksum slot as warm by measuring the gas used by `sload` and comparing it with `_WARM_READ_THRESHOLD`, which is fixed at 1000 gas. This matches the reviewed Cancun/EIP-2929 behavior because warm storage reads are far below the threshold and cold reads are above it.

The check is sensitive to future gas repricing. If a hard fork changes storage-read gas costs or access-list accounting, the fixed threshold could misclassify warm or cold checksum slots. A false positive would weaken message validation. A false negative would break valid interop messages.

Recommendation: Treat the threshold as fork-sensitive protocol logic. Every network upgrade that changes storage gas accounting should include regression tests for warm and cold checksum slots. If future gas schedules narrow the gap around the current threshold, update `_WARM_READ_THRESHOLD` before activating the fork.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.8 `notEntered` requires an entered call-depth context

Severity: Informational

Context: `TransientContext.sol#L93-L98`

Description: `notEntered` reverts when `TransientContext.callDepth()` is zero. The modifier therefore requires an entered call-depth context even though its name suggests that it checks for the opposite condition.

The Natspec also describes cross-domain sender and source metadata, but the modifier only reads the generic call-depth slot. The modifier is unused today, so this is not an active vulnerability. It can still lead to future misuse if a maintainer applies it expecting a "not entered" guard.

Recommendation: Rename the modifier to describe the actual check, for example `onlyCallDepthEntered`, or remove it until it has a production use. Update the Natspec so it describes call-depth state instead of cross-domain message metadata.

OP Labs: Acknowledged. Could be removed as part of a cleanup along with other similar unused parts.

Cantina Managed: Acknowledged.

3.2.9 `CrossL2Inbox` target filtering does not match documentation

Severity: Informational

Context: `L2ToL2CrossDomainMessenger.sol#L144-L145`

Description: The messenger only rejects `Predeploys.L2_T0_L2_CROSS_DOMAIN_MESSENGER` as a target. Local comments and tests describe `CrossL2Inbox` as an unsafe target too, but the implementation does not reject it.

We did not identify a direct exploit from sending a message to `CrossL2Inbox`. `CrossL2Inbox` is already permissionless and still requires the checksum access-list validation. The concern is that integrators and relayers may rely on documentation that describes stricter target filtering than the contract actually performs.

Recommendation: Either update the implementation to reject `Predeploys.CROSS_L2_INBOX` or update the `sendMessage` NatSpec at `src/L2/L2ToL2CrossDomainMessenger.sol:128-130` and the send-message test comment/assumption at `test/L2/L2ToL2CrossDomainMessenger.t.sol:195-196` to state that only `Predeploys.L2_T0_L2_CROSS_DOMAIN_MESSENGER` is blocked.

OP Labs: Fixed in commit `7c2be5c`.

Cantina Managed: Fix verified. The tests and documentation now match the implementation: only the L2-to-L2 messenger predeploy is treated as an unsafe send target and `CrossL2Inbox` is no longer described as blocked.

3.2.10 `ReentrantAware` can leave call depth incremented after inline assembly termination

Severity: Informational

Context: `TransientContext.sol#L74-L79`

Description: `reentrantAware` increments the transient call depth before the modified function body and decrements it after the body returns. This is balanced for ordinary Solidity returns and for reverts. It is not balanced if a future inheriting function exits with EVM-level `return` or `stop` from inline assembly, because those opcodes terminate the call frame before the modifier epilogue runs.

Current production code does not call `reentrantAware`; `L2ToL2CrossDomainMessenger` inherits `TransientReentrancyAware` but uses `nonReentrant`. Therefore this is not an active production bug. It is a future inheritance hazard. If a future contract uses `reentrantAware` around generic `TransientContext.get` or `set` state, a skipped decrement can leave the call depth nonzero for the rest of the transaction and shift later calls into the wrong depth namespace.

Recommendation: Document that `reentrantAware` must not wrap functions that can execute inline assembly `return` or `stop`. Keep the modifier unused unless the wrapped function has ordinary Solidity control flow.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.11 Call-depth wraparound comment overstates safety

Severity: Informational

Context: `TransientContext.sol#L41-L58`

Description: `TransientContext.increment` and `TransientContext.decrement` update the call-depth slot with assembly `add` and `sub`, so overflow and underflow wrap instead of reverting. The comments say this is acceptable because there is still only one value stored per slot.

That explanation is too broad. `TransientContext.get` and `TransientContext.set` do not store values directly under the caller-provided slot. They derive the transient storage key from `(callDepth, slot)`. If call depth wraps, the library reuses an earlier depth namespace, so a write at the wrapped depth can overwrite a value that belonged to another depth.

This is not exploitable through the current in-scope production contracts. The helpers are internal, production code does not expose raw call-depth changes and balanced `reentrantAware` usage cannot practically overflow a `uint256` within EVM call-depth limits. The issue is a future-use hazard: the comment makes wraparound sound harmless when correctness actually depends on balanced enter/exit behavior and practical non-reachability of wraparound.

Recommendation: Update the comments on `increment` and `decrement` to say that correctness depends on balanced use of the call-depth helpers and the practical impossibility of `uint256` wraparound under normal EVM execution. Avoid exposing these helpers or wrappers around them, through public or external functions unless callers cannot create unbalanced depth changes.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.12 `ETHLiquidity.mint` underfunding has a generic revert

Severity: Informational

Context: `ETHLiquidity.sol#L43-L49`

Description: `ETHLiquidity.mint` does not check that `_amount` is less than or equal to `address(this).balance` before it deploys `SafeSend` with `_amount` wei. If the reservoir is underfunded, the function reverts during the value transfer to `new SafeSend` instead of returning a domain-specific liquidity error.

This does not let an attacker mint unbacked ETH. The revert rolls back the bridge relay and the message remains retryable at the messenger layer. The concern is operational: liquidity misconfiguration or unexpected depletion is harder for relayers, monitors and tests to classify because the failure is a generic low-level creation failure.

Recommendation: Add an explicit insufficient-balance check before `SafeSend` is created:

```
if (_amount > address(this).balance) revert InsufficientLiquidity();
```

OP Labs: Acknowledged. This doesn't cause an issue since the call will revert. `ETHLiquidity` should be prefunded on interop activation and reaching the described state should not be possible.

Cantina Managed: Acknowledged.

3.2.13 L2-to-L2 relay does not validate message version

Severity: Informational

Context: [L2ToL2CrossDomainMessenger.sol#L221-L235](#)

Description: `messageNonce` reserves the upper two bytes of the nonce for a message version and the current `messageVersion` is zero. `relayMessage` decodes the nonce from the validated `SentMessage` payload and includes it in `hashL2toL2CrossDomainMessage`, but it does not reject nonces whose version bits are unknown or unsupported.

The current `sendMessage` implementation only emits version-zero nonces, so this is not reachable through the honest source messenger. The concern is fail-closed compatibility. If a future or mismatched source messenger emits a higher-version `SentMessage` with the same event shape, an older destination messenger will relay it instead of rejecting the unsupported version.

Recommendation: Decode the version bits from the relayed nonce and reject unsupported versions before computing the message hash or calling the target.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.14 `onlyEntered` relies on the reentrancy guard instead of metadata state

Severity: Informational

Context: [TransientContext.sol#L100-L105](#)

Description: The cross-domain context getters are supposed to answer one question: "is a message currently being delivered, and if so, who sent it and from which chain?" Today they answer a different question. `onlyEntered` only checks whether `_entered()` is true, and `_entered()` is controlled by the generic `nonReentrant` guard.

That is a weaker check than "message metadata has been written." In `relayMessage`, `nonReentrant` turns the guard on before `CrossL2Inbox.validateMessage` runs. The messenger only stores the cross-domain sender and source later, after validation and replay checks. This means there is a short period where `_entered()` is true but the message context is still empty.

In the current production code, that window is not exploitable because `CrossL2Inbox.validateMessage` does not call attacker-controlled code. The concern is future coupling. If a later change adds a callback-capable dependency before `_storeMessageMetadata`, or if another function reuses `nonReentrant` and then calls a context getter, the getter can return zero or stale metadata instead of reverting.

Put more simply: the code uses the "do not reenter me" flag as if it were also the "message context is ready" flag. Those are not the same state.

For example, assume a future version wraps `CrossL2Inbox.validateMessage` with an adapter that can call back into the messenger before `_storeMessageMetadata(source, sender)` runs. During that callback, `ENTERED_SLOT` is already set by `nonReentrant`, so `crossDomainMessageContext()` passes `onlyEntered`. However, `CROSS_DOMAIN_MESSAGE_SENDER_SLOT` and `CROSS_DOMAIN_MESSAGE_SOURCE_SLOT` have not been written yet, so the callback reads `(address(0), 0)` instead of reverting.

The same issue appears if a future unrelated function is marked `nonReentrant` and calls `crossDomainMessageContext()` without relaying a message. The reentrancy guard is active, so the getter succeeds even though no cross-domain message context exists.

Recommendation: Use a dedicated transient metadata-active flag for the cross-domain context getters. Set the flag in the same helper that writes sender and source, then clear it when the metadata is cleared. Alternatively, document that `nonReentrant` must not be reused for unrelated functions without re-auditing the context getters.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.15 Output root version is not constrained

Severity: Informational

Context: [Hashing.sol#L111-L124](#)

Description: `Hashing.hashOutputRootProof` hashes the `version` field but does not validate it. Any `bytes32` version is accepted and included in the output-root preimage.

The Go-side comparison used in the tests is stricter. The local Forge FFI helper at `packages/contracts-bedrock/scripts/go-ffi/utils.go:153`, `hashOutputRootProof(...)`, is only a wrapper. It builds a `bindings.TypesOutputRootProof` and delegates to `op-node/rollup/output_root.go:13`, `ComputeL2OutputRoot(...)`.

That delegated Go function rejects any version other than `eth.OutputVersionV0` before computing the root. The Solidity helper does not do the same check. It only commits the supplied version into the hash.

No direct exploit was found in the current code as the reviewed onchain callers compare the computed hash against an already accepted root claim, so an unsupported version still needs to match the committed claim. The risk is future misuse: a new caller could treat `hashOutputRootProof` as a validation helper and assume it rejects unsupported output-root versions. It does not. It only hashes whatever `version` value the caller provides.

Recommendation: Either enforce the supported output-root version before hashing, or document that callers must validate the version separately.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.16 Prefunded SafeSend addresses inject extra value

Severity: Informational

Context: [SafeSend.sol#L8-L10](#)

Description: `SafeSend` transfers the temporary contract's full balance with `selfdestruct(_recipient)`. The temporary contract address is deterministic from the creator address and nonce, so anyone can send ETH or native asset to a future `SafeSend` address before the production caller executes `new SafeSend`.

When the caller later creates `SafeSend`, the constructor sweeps both the caller's endowment and the prefunded balance. In one-step callers such as `Faucet.withdraw` or the final `SuperchainETHBridge.relayETH` delivery, the recipient receives more than the event amount. In two-step callers such as `SuperchainETHBridge.relayETH` and `LiquidityController.mint`, prefunding the first temporary address can also leave attacker-funded dust in the intermediate bridge or controller contract while events still report only `_amount`.

Recommendation: Document that `SafeSend` delivers the full temporary account balance, not only the value supplied by the caller. If any caller requires exact amount delivery or a zero residual balance invariant, replace `SafeSend` with a primitive that cannot sweep prefunded temporary-address balances, or add explicit operational handling for unexpected dust in the caller.

OP Labs: Fixed in commit [9d7f24d](#).

Cantina Managed: Fix verified. `SafeSend` now documents that `selfdestruct` transfers the temporary contract's entire balance, not just the caller-supplied value, so callers and integrators are warned about prefunded temporary-address balances.

3.2.17 ResendMessage can spam duplicate valid message logs

Severity: Informational

Context: [L2ToL2CrossDomainMessenger.sol#L163-L195](#)

Description: `resendMessage` is permissionless and emits `SentMessage` again for any message whose parameters match a stored `sentMessages` hash. It does not track how many times a message has been re-emitted and it cannot know whether the message already succeeded on the destination chain.

This does not bypass destination replay protection. `successfulMessages` prevents a successfully relayed message from executing again. The issue is operational: a sender can repeatedly emit valid-looking duplicate logs for the same message and force naive relayers or indexers to spend resources deduplicating them.

The source-chain caller pays for the duplicate logs, so this is bounded by source-chain gas costs.

Recommendation: Relayers and indexers should deduplicate by the L2-to-L2 message hash and destination relay status instead of treating each `SentMessage` log as a fresh executable command.

OP Labs: Acknowledged. Relayers and indexers should be able to filter spam.

Cantina Managed: Acknowledged.

3.2.18 `sendETH` accepts zero-value bridge messages

Severity: Informational

Context: [SuperchainETHBridge.sol#L41-L58](#)

Description: `sendETH` rejects a zero recipient but does not reject `msg.value == 0`. A caller can therefore create a valid `SendETH` event and messenger record for a bridge transfer that has no ETH behind it.

The destination relay then executes `relayETH` with `_amount == 0`. This does not steal funds or corrupt `ETHLiquidity`, but it still creates source-chain bridge logs and destination-chain relay work. Consequently, relayers, indexers and monitors must handle zero-value bridge messages even though they cannot deliver value to a recipient.

Recommendation: If zero-value ETH bridge messages are not intentionally supported, reject them before calling `ETHLiquidity.burn`.

```
if (msg.value == 0) revert InvalidAmount();
```

If zero-value messages are intentional, document them and ensure relayers and indexers filter them as no-value bridge operations.

OP Labs: Acknowledged. "Noise" of potential spam calls already exists with `sendMessage` as well.

Cantina Managed: Acknowledged.

3.2.19 Hand-maintained `SentMessage` selector can drift from the event layout

Severity: Informational

Context: [L2ToL2CrossDomainMessenger.sol#L60-L61](#)

Description: `L2ToL2CrossDomainMessenger` validates relayed payloads against a manually maintained `SENT_MESSAGE_EVENT_SELECTOR`, then decodes the payload according to the current `SentMessage` log layout: selector plus indexed topics, followed by ABI-encoded event data.

The selector is not derived from `SentMessage.selector` in-contract, and the decoder is separately coupled to the event's indexed/non-indexed field layout. If the event signature or indexed fields are

changed without updating the constant and decoder together, valid messages could become unrelayable, or relayers could decode payloads inconsistently with the emitted log format.

Recommendation: Add an explicit test or build-time check that `SENT_MESSAGE_EVENT_SELECTOR == L2ToL2CrossDomainMessenger.SentMessage.selector` and that an emitted `SentMessage` log round-trips through `_decodeSentMessagePayload`. Once tooling allows it, replace the literal selector with direct use of the event selector, and keep the expected log payload layout documented/tested.

OP Labs: Acknowledged. We believe there is a sufficient test that would fail in the case the indexed/non-indexed layout changes in the `op-acceptance-tests/tests/supernode/interop/eth_bridge_test.go` file.

Cantina Managed: Acknowledged.

3.2.20 `ETHLiquidity` invariant checks the wrong balance

Severity: Informational

Context: `ETHLiquidity.t.sol#L34`

Description: The `ETHLiquidity` invariant test intends to ensure repeated mint/burn operations cannot cause the modeled actor's balance to exceed the expected `uint248.max` liquidity bound. However, the test setup funds `Predeploys.SUPERCHAIN_ETH_BRIDGE`, while the invariant asserts `address(actor).balance`.

During `mint()`, the actor pranks as `SUPERCHAIN_ETH_BRIDGE`, so ETH is released to the bridge predeploy, not to the actor.

As a result, the invariant can pass while failing to exercise the intended balance safety property. This is a test coverage gap rather than a direct protocol vulnerability.

Recommendation: Update the invariant model to check the balances that actually move in the modeled flow: `Predeploys.ETH_LIQUIDITY` and `Predeploys.SUPERCHAIN_ETH_BRIDGE`, not `address(actor)`.

OP Labs: Fixed in commit `6a580e4`.

Cantina Managed: Fix verified. The invariant now tracks the actual balances that move in the model: ETH released from `ETHLiquidity`, ETH locked by the bridge, and ETH added through `fund`, then asserts conservation against `address(ethLiquidity).balance` instead of `address(actor).balance`.

3.2.21 `ETHLiquidity` events record only `msg.sender` (always the bridge), losing the initiator

Severity: Informational

Context: `ETHLiquidity.sol#L38-L49`

Description: `ETHLiquidity` emits `LiquidityBurned` and `LiquidityMinted` with only `msg.sender` and the amount. For normal Superchain ETH bridge flows, `msg.sender` is always `Predeploys.SUPERCHAIN_ETH_BRIDGE`, because `SuperchainETHBridge` is the only authorized caller of `burn()` and `mint()`.

As a result, the `ETHLiquidity` events describe aggregate liquidity movement but do not identify the original sender, recipient, source chain, or destination chain. User-level attribution is only available by correlating with `SuperchainETHBridge`'s `SendETH` and `RelayETH` events.

Recommendation: If `ETHLiquidity` logs are intended to be independently useful for monitoring, include additional context (original sender) in the emitted events.

OP Labs: Acknowledged. `ETHLiquidity` events are used more for accounting; `SendETH` and `RelayETH` should be sufficient.

Cantina Managed: Acknowledged.

3.2.22 `L2ToL2CrossDomainMessenger` `sendMessage` documentation contains stale legacy messenger warnings

Severity: Informational

Context: `L2ToL2CrossDomainMessenger.sol#L128-L130`

Description: `L2ToL2CrossDomainMessenger.sendMessage` includes NatSpec copied from the legacy `CrossDomainMessenger` path. The comment warns that “any ETH sent will be permanently locked” if the destination call is unrelayable, and says the same can occur when the target is considered unsafe by `_isUnsafeTarget()`.

Both statements are stale for this contract. `L2ToL2CrossDomainMessenger.sendMessage` is not payable, so users cannot attach ETH at send time. The contract also does not implement `_isUnsafeTarget()`; it only rejects `_target == Predeploys.L2_TO_L2_CROSS_DOMAIN_MESSENGER`.

The same stale text is duplicated in `IL2ToL2CrossDomainMessenger`. This can mislead integrators about the L2-to-L2 messenger’s ETH semantics and target filtering behavior.

Recommendation: Update the NatSpec in both `L2ToL2CrossDomainMessenger` and `IL2ToL2CrossDomainMessenger` to describe the actual behavior.

OP Labs: Fixed in commit `53d21dd`.

Cantina Managed: Fix verified. The stale legacy messenger warnings were removed from both the implementation and interface NatSpec, which now correctly state that `sendMessage` is nonpayable, cannot send ETH and blocks only the L2-to-L2 messenger target.

3.2.23 `L2ToL2CrossDomainMessenger` redeclares errors inherited from `TransientReentrancyAware`

Severity: Informational

Context: `L2ToL2CrossDomainMessenger.sol#L14-L15`, `L2ToL2CrossDomainMessenger.sol#L35-L36`

Description: `L2ToL2CrossDomainMessenger` declares `NotEntered()` and `ReentrantCall()` at file scope, and `IL2ToL2CrossDomainMessenger` declares the same errors in the interface. However, the actual revert paths for these errors are inherited from `TransientReentrancyAware`.

`ReentrantCall()` is emitted by the inherited `nonReentrant` modifier, while `NotEntered()` is emitted by the inherited `onlyEntered` / `notEntered` modifiers. The local declarations in `L2ToL2CrossDomainMessenger` are therefore redundant and do not define the code paths that actually revert.

Recommendation: Remove the redundant `NotEntered()` and `ReentrantCall()` declarations from `L2ToL2CrossDomainMessenger`.

OP Labs: Fixed in commit `2c5029b`.

Cantina Managed: Fix verified. The implementation no longer redeclares `NotEntered()` or `ReentrantCall()`; revert selectors now come from `TransientReentrancyAware` and the tests assert the inherited selectors.

3.2.24 `CrossL2Inbox` validation failure error differs from the spec reference implementation

Severity: Informational

Context: `CrossL2Inbox.sol#L79`

Description: The OP Stack interop spec's reference implementation for `CrossL2Inbox.validateMessage` reverts with `NonDeclaredExecutingMessage()` when the checksum storage slot is not warm.

However, the deployed implementation declares and reverts with `NotInAccessList()`. Because custom error names determine the revert selector, `NonDeclaredExecutingMessage()` and `NotInAccessList()` are not ABI-equivalent.

Recommendation: Update the spec reference implementation to use `NotInAccessList()`, matching the contract and interface, or intentionally rename the contract/interface error if `NonDeclaredExecutingMessage()` is meant to be canonical.

OP Labs: Fixed in PR 916.

Cantina Managed: Fix verified.

3.2.25 `CrossL2Inbox` spec requires `Identifier` getters that are not implemented by the stateless design

Severity: Informational

Context: `CrossL2Inbox.sol#L31`

Description: The interop predeploys spec states that the `Identifier` "MUST be exposed via public getters so that contracts can call back to authenticate properties about the `_msg`".

However, `CrossL2Inbox` does not expose any public getters for the current `Identifier`. Its implementation is stateless: callers pass the `Identifier` directly into `validateMessage`, the contract verifies the checksum access-list slot, and then emits `ExecutingMessage`. This appears to be stale spec text rather than a missing implementation requirement.

Recommendation: Update the spec to remove the `Identifier` getter requirement from `CrossL2Inbox`. If getter-style context is intended only for `L2ToL2CrossDomainMessenger`, move the requirement to that section and reference `crossDomainMessageSender()` / `crossDomainMessageSource()` explicitly.

OP Labs: Fixed in PR 916.

Cantina Managed: Fix verified.